DevSpace — Project Blueprint v3

Sep 24, 2026 · @Ayush

Overview

Core idea: a leader creates a project, team members each work on their own copy in a browser IDE inside DevSpace, they submit their work for review, and only leaders merge it into main, which can be published to GitHub.

Every feature must serve that sentence. If it doesn't, it waits or is dropped.

DevSpace is a web workspace: log in, pick a team and project, click **Open IDE**, and real VS Code opens inside DevSpace with your copy loaded. Git runs underneath but members never type Git commands. That is the difference from GitHub.

Fixed rules that no setting can switch off:

- Nobody pushes directly to main; main changes only through a merged submission.
- Members change only their own copy (the one exception is a leader helping with a conflict, which is logged).
- Nobody approves their own submission.
- No feature may pretend to work: unfinished code returns "not implemented", never fake success.

Audience and goals

DevSpace is for small teams of 2 to 10 people: student groups, hackathon and bootcamp teams, and small startups.

- **Simple first:** a team with no Git experience can start in minutes.
- **Production quality:** real security, real tests and no mocks, because the goal is both a portfolio project and a launchable product.
- **Tight scope:** a small product that fully works beats a large one that partly works.

Roles, permissions and settings

A project has one Leader, any number of Co-leaders, and Members whose extra powers the leaders switch on individually.

Role	Can do
Leader (owner)	Everything, including deleting the project, transferring ownership, adding and removing co-leaders
Co-leader	Everything except deleting the project, transferring ownership, or removing the leader or other co-leaders
Member	Work on and submit from their own copy, plus any permissions switched on below

Permissions a leader can switch on per member:

Permission	Allows
Approve submissions	Approve others' work (approving is not merging)
Comment	Comment and suggest changes on others' submissions
Create tasks	Create tasks and assign anyone
AI cross-review	Run automatic AI cross-review on submissions (free models by default)
Publish to GitHub	Publish or sync the project with GitHub

Leaders and co-leaders always have every permission, and only they can merge, reject, undo merges and help fix conflicts.

Project settings, chosen by the leader:

Setting	Options
Require approval before merge	On (at least one approval) or off
Members can browse others' copies	On or off; submitted work is always visible
Update copies after merge	Automatic or manual ("Update my copy" button)
GitHub mode	Publish once, or keep updated

Core workflow

Work moves one way: from a member's copy, through review, into main.

```
flowchart TD
    A[Leader creates project<br/>empty or from GitHub] --> B[Invites members,
<br/>assigns tasks]
    B --> C[Member clicks Open IDE<br/>VS Code with their copy]
    C --> D[Codes, runs, tests]
    D --> E[Submits all or<br/>selected changed files]
    E --> F[Team comments,<br/>AI cross-review]
    F --> G{Leader or<br/>co-leader}
    G -->|Request changes| D
    G -->|Reject with reason| H[Rejected]
    G -->|Merge| I[Main updated]
    I --> J[Copies updated<br/>auto or manual]
    I --> K[GitHub updated<br/>if enabled]
```

If approval is required, a submission needs at least one approval from someone other than its author before the merge button works.

Copies and the browser IDE

Each member has exactly one copy per project, and clicking **Open IDE** opens it in real VS Code inside the DevSpace page.

What happens on Open IDE:

1. DevSpace starts a Docker container for that copy, or reuses it if already running.
2. The container holds the member's files (a hidden Git working copy) and runs OpenVSCode Server.
3. The IDE appears inside DevSpace with editor, terminal and extensions, so members can run and test code.
4. After 30 minutes idle the container stops; files are kept and the next Open IDE resumes.

Every container:

- is isolated: it can reach only its own copy, never other members' files or the host;
- has CPU and memory limits;

- has DevSpace's own tracker extension pre-installed (reports the open file and activity for presence), and members add any AI tool they like, logged in with their own account;
- gets extensions from Open VSX, so a few Microsoft-only extensions are unavailable.

Updating a copy after a merge: in automatic mode DevSpace merges the new main into the copy when it can do so cleanly. If it conflicts with the member's edits, the copy is left untouched and the member is told to update and resolve it in the IDE. Work is never overwritten silently.

Later, not in the first version: opening the same copy in desktop VS Code or JetBrains.

Submissions and review

A submission is a snapshot of chosen changed files from a member's copy, compared against main.

Submitting: DevSpace lists every file changed versus main, all ticked. The member submits all of them or unticks some, adds a title and description, and can link a task. The copy keeps working; later edits don't enter the submission unless the member clicks **Update submission**.

Several open submissions are allowed, but one file can be in only one of the same member's open submissions at a time.

Review page: file-by-file diff against main. People with permission can comment on lines, suggest a fix the author accepts or rejects, approve, or request changes.

Status	Meaning
Open	Waiting for review
Changes requested	Author must update it
Approved	Ready to merge
Needs update	Conflicts with main; author must resolve
Merged	In main
Rejected	Refused by a leader, with a written reason; work stays in the copy
Closed	Cancelled by the author or a leader

Status	Meaning
Reverted	Merged, then undone by a leader

Merging, conflicts and undo

Leaders and co-leaders merge; each merged submission becomes one commit on main, titled with the submission and credited to its author.

- **Merge is allowed** only when there are no conflicts and, if the project requires it, at least one approval from someone other than the author.
- **Conflicts:** the submission shows Needs update. The author updates their copy and resolves it in the IDE. If stuck, a leader or co-leader can open the author's copy and fix it; this is logged in activity and the author is notified.
- **Undo:** a leader can revert a merged submission. DevSpace adds a new commit that reverses it rather than rewriting history, marks the submission Reverted, and the work remains in the author's copy.
- **Done means done:** after a merge, the file in main must actually contain the change. Tests check the files, not only the status.

Tasks

Leaders, and members with the Create tasks permission, create tasks with a title, description, one or more assignees and an optional due date.

- Statuses: **To do → In progress → In review → Done.**
- A task moves to In review when any linked submission opens.
- After a linked submission merges, DevSpace asks a leader "Mark this task done?"; the leader decides, since a task may need several people's work.
- Leaders can always edit or reassign any task.
- Everyone sees the task board.

Presence and notifications

Everyone in a project sees each teammate's status and the file they are editing; statuses cannot be hidden.

Status	Meaning
Active	Typed in the IDE or clicked in DevSpace in the last 5 minutes; shows the open file, e.g. editing `src/login.tsx`
Online	DevSpace open, idle over 5 minutes
Offline	DevSpace closed; shows "last seen" time

Presence is visible only to members of the same project and is stored in Redis with a short expiry.

Notifications are in-app only in the first version: submissions to review, reviews and comments on your work, merges, rejections, conflicts, task assignments, and a leader editing your copy.

AI

Every member can use AI for free, anyone can optionally upgrade to stronger models on their own account, and nobody ever pays for anyone else.

Members never paste API keys into DevSpace, and DevSpace never uses anyone's ChatGPT, Claude or Gemini login (Anthropic, for example, forbids third-party apps from using Claude.ai logins).

1. AI help while coding (in the IDE). Members use any AI tool they like inside their own VS Code (official Claude, Gemini, ChatGPT or other extensions), logged in with their own account and plan, free or paid. DevSpace does not touch these logins.

2. Automatic AI cross-review. A member clicks **Connect with OpenRouter** once (a login flow; DevSpace receives a key the member controls and can revoke). On any submission, **Cross-review with AI** runs several AI models on the diff in the background and shows findings side by side, highlighting issues several models agree on.

Plan	What it uses	Cost
Free (default)	OpenRouter free models, with automatic fallback if one is busy or removed	$0, no card; about 50 requests a day (roughly 15 to 25 reviews)

Plan	What it uses	Cost
Upgrade (optional)	Paid models such as Claude or GPT on the member's own OpenRouter account	Paid by that member only

3. Review with my IDE AI. A button on every submission opens the reviewer's IDE with the changes and a ready-made review instruction. The reviewer asks whatever AI they already use; its findings are saved back to the submission. Free for DevSpace, and it works with any AI plan the member already has.

Rules: humans always decide, AI never approves or merges; AI jobs run in the background worker; every AI review records source (automatic or IDE), model and whether it was free or paid.

Later: DevSpace's own AI panel and a project assistant that summarizes team activity.

GitHub

DevSpace is the source of truth for main; GitHub is connected in both directions, but nothing reaches main without a leader's merge.

- **Import:** a new project starts empty or from an existing GitHub repository.
- **Publish:** leaders, or members with the Publish to GitHub permission, connect their GitHub account, pick a new or empty repo, public or private, and DevSpace pushes main with full history.
- **Keep updated mode:** every merge is pushed automatically using the connection of whoever set it up.
- **Incoming changes (later phase):** changes made directly on GitHub arrive as a submission labelled "Incoming from GitHub" that a leader reviews and merges like any other. Until pulled in, auto-publish pauses and the leader is told, so nothing is overwritten.
- Uses a GitHub App with real API calls; tokens are stored encrypted, never discarded or faked.

Scope

Anything not in the first-version list waits until the core works; anything "not planned" is dropped unless we explicitly revisit it.

Area	First version	Later	Not planned
Accounts, teams, projects	Yes		
Leader, co-leader, member roles and permissions	Yes		
Tasks	Yes		
Copies with browser VS Code	Yes		
Submit, review, merge, conflicts, undo	Yes		
Presence (with open file)	Yes		
In-app notifications	Yes		
AI in the IDE with members' own logins	Yes		
AI cross-review: free models, optional paid upgrade, Review with my IDE AI	Yes		
Import from and publish to GitHub	Yes		
Two-way GitHub sync		Yes	
DevSpace AI panel, project assistant		Yes	
Desktop VS Code / JetBrains connection		Yes	
Email notifications		Yes	
Team chat		Yes	
Live co-editing of one file			Yes
Autonomous AI coding agent			Yes
Payments and billing			Yes
Mobile app			Yes

Architecture and tech stack

DevSpace is a modular monolith: a web app, an API server, a background worker, and one IDE container per open copy, all in TypeScript.

```
flowchart LR
    U[Browser] --> W[Web app<br/>React]
    W --> API[API server<br/>Express + Socket.io]
    W -.->|IDE iframe via proxy| API
    API --> PG[(PostgreSQL)]
    API --> R[(Redis)]
    API --> G[Git repos<br/>on disk]
    API --> D[Docker Engine]
    D --> C[IDE containers<br/>OpenVSCode Server]
    C --> G
    R --> WK[Worker<br/>BullMQ]
    WK --> AI[AI providers]
    WK --> GH[GitHub]
```

The API also acts as a proxy for IDE traffic, checking the user may open that copy before passing requests to its container. Slow work (AI reviews, GitHub sync) runs in the worker.

Layer	Technology
Frontend	React, TypeScript, Vite, React Router, TanStack Query, Zustand, Tailwind CSS
Diff view	Monaco diff editor (read-only, on review pages)
Backend	Node.js, TypeScript, Express, Zod
Database	PostgreSQL + Prisma
Auth	HTTP-only cookie sessions stored in PostgreSQL
Git	Real `git` on the server: one bare repo per project, one working copy per member
IDE	OpenVSCode Server in Docker, managed via the Docker Engine API (dockerode)
Realtime	Socket.io + Redis adapter
Jobs	BullMQ + Redis, separate worker process
AI	OpenRouter via its login flow (OAuth PKCE), OpenAI-compatible client, free models with fallbacks
GitHub	GitHub App

Layer	Technology
Tooling	npm workspaces, ESLint, Prettier, Vitest, GitHub Actions CI
Dev setup	Docker Compose for PostgreSQL and Redis (no hand-built Redis)

Data model

Git stores all code; PostgreSQL stores people, permissions, workflow and commit references, never file contents.

Table	Key fields
users	id, name, email, password_hash
sessions	id, user_id, expires_at
teams	id, name, owner_id
team_members	team_id, user_id
projects	id, team_id, name, repo_path, settings (JSON)
project_members	project_id, user_id, role (leader, co_leader, member), permissions (JSON)
invitations	id, project_id, email, token_hash, expires_at, status
copies	id, project_id, user_id, branch_name, container_id, container_status, last_active_at
tasks	id, project_id, title, description, status, due_date
task_assignees	task_id, user_id
submissions	id, copy_id, task_id, title, base_sha, head_sha, status, reject_reason, merged_sha
submission_files	submission_id, file_path
reviews	id, submission_id, reviewer_id, decision

Table	Key fields
comments	id, submission_id, author_id, file_path, line, body, suggestion
ai_connections	id, user_id, provider (openrouter), encrypted_key, connected_at
ai_reviews	id, submission_id, requested_by, source (auto or ide), model, is_free, findings (JSON)
notifications	id, user_id, type, data, read_at
activity_log	id, project_id, actor_id, action, target, data, created_at (append-only)
github_links	project_id, installation_id, repo_full_name, mode, set_up_by

Presence lives in Redis with a short expiry, not PostgreSQL.

Build phases

Twelve phases deliver the first version; each ends with something you can see working in the browser, checked by a real end-to-end test.

Phase	Builds	Done when
1. Foundation	Monorepo, apps, Prisma, Docker Compose, lint, Vitest, CI	`npm run dev` starts web and API; CI is green
2. Accounts	Sign up, log in, log out, sessions	A user signs up, logs in and stays logged in after refresh
3. Teams and projects	Teams, projects, invitations, roles, permissions, settings	A leader invites a member, who joins and sees the project
4. Project repository	Bare Git repo per project, start empty or from a public Git URL, read-only file browser	The project's files show in the browser

Phase	Builds	Done when
5. Copies and IDE	Copy per member, IDE container, proxy, iframe, idle stop, isolation	Member clicks Open IDE, edits and runs code in the terminal; another member cannot reach that container
6. Submissions and review	Choose files, submit, diff, comments, suggestions, approve, request changes, reject	A member submits selected files; a reviewer comments and approves
7. Merge	Merge, conflicts, Needs update, leader help, undo, copy updates	After merge the file in main really changes; revert restores it
8. Tasks	Task board, assignees, links to submissions	A task moves to In review on submit and Done after the leader confirms
9. Presence and notifications	Tracker extension, statuses, open file, in-app notifications	Teammates see "editing src/app.ts" live
10. AI	OpenRouter login, free-model cross-review, Review with my IDE AI	A member with no paid plan gets two free AI reviews of a submission
11. GitHub	GitHub App, import, publish once, keep updated	The project appears on GitHub and updates after a merge
12. Deploy	Hosting, HTTPS, logging, backups, rate limits	A live demo team can use it end to end

Phase 5 is the hardest and will be split into small steps. Later phases after the first version: two-way GitHub sync, AI panel and assistant, desktop IDE connection, email, chat.

Rules for building and open questions

These rules exist so the project never drifts from this blueprint again.

1. This doc is the source of truth. Any change to the idea is agreed and written here first, then built.

2. One phase at a time. A phase is finished only when its "Done when" works in the browser and its end-to-end test passes.
3. Tests check real results (files in main, containers isolated), not just statuses.
4. No mocks presented as finished work.
5. Commit to Git after every working step; push to a private GitHub repo.
6. Read and understand each phase before starting the next; record big choices in `docs/decisions.md`.

Open questions to settle when their phase starts:

- [ ] Where to host the deployed version, and its server size for IDE containers (Phase 12)
- [ ] Which free OpenRouter models to use by default and as fallbacks (Phase 10)
- [ ] Default container limits: CPU, memory, disk per copy (Phase 5)
- [ ] Whether a team can hold several projects with different leaders (Phase 3; assumed yes)